

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

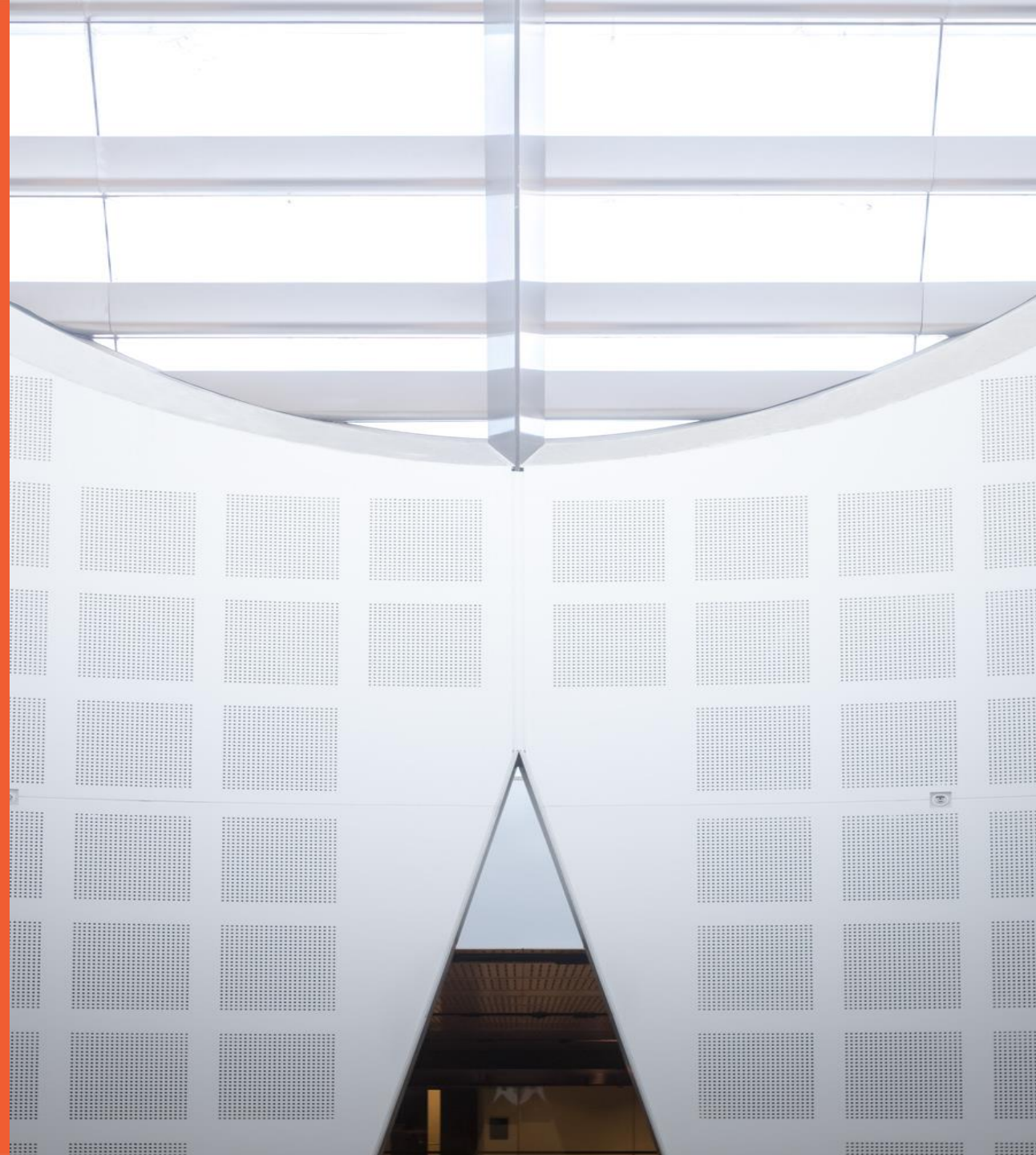
Do not remove this notice.

COMPx270: Randomised and
Advanced Algorithms
Lecture 8: Streaming and
Sketching I

Clément Canonne
School of Computer Science



THE UNIVERSITY OF
SYDNEY



A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything done, **only your memory**. **What is its average degree?**

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything done, **only your memory**. **What is its average degree?**

(1,2)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(2,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything done, **only your memory**. **What is its average degree?**

(1,2)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(4,5)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(4,5)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(3,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(3,6)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(1,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(4,6)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything done, **only your memory**. **What is its average degree?**

(3,5)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(3,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(4,5)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(3,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(3,6)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

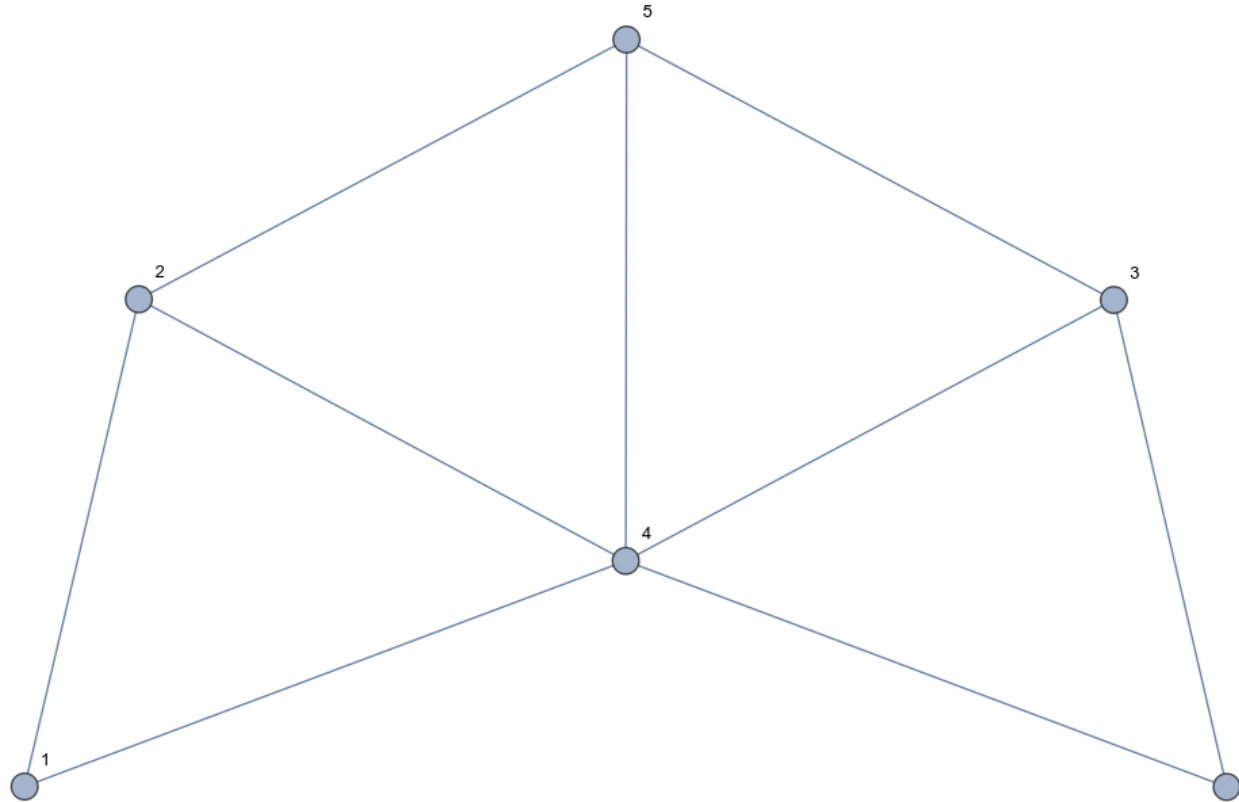
(1,4)

A question

You have a graph, coming one edge at a time, with possible duplicates, and no paper to write anything down, **only your memory**. **What is its average degree?**

(4,6)

A question (an answer)



Streaming algorithms: what? (1/3)

- **Low memory:** cannot store whole input
- Input comes as a **stream**: sequence of length **m**

$$\sigma = (a_1, a_2, \dots, a_m)$$

of elements from universe of size **n**

- **Worst-case** (arbitrary) **order**

Sublinear space: $s = o(\min(m, n))$ bits

Streaming algorithms: what? (2/3)

- Randomised algorithms
- Approximate: typically, want to compute some value $v \geq 0$, but we're fine with

$$\hat{v} \approx_{\varepsilon} v$$

- Multiplicative approximation:

$$\Pr[|\hat{v} - v| \geq \varepsilon v] \leq \delta$$

- Additive approximation:

$$\Pr[|\hat{v} - v| \geq \varepsilon] \leq \delta$$

Streaming algorithms: what? (3/3)

- Number of passes: p **passes** over the stream σ .
- Type of updates: **cash register** versus **turnstile** setting

First example: Majority

First example: Majority (Frequency Estimation)

First example: the Misra-Gries algorithm (1/4)

Input: Parameter $\varepsilon \in (0, 1]$

```
1:  $A \leftarrow \emptyset$ 
2: Set  $k \leftarrow \lceil 1/\varepsilon \rceil$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i = j \in [n]$ 
5:   if  $A[j] > 0$  then
6:      $A[j] \leftarrow A[j] + 1$ 
7:   else if  $A[j] = 0$  and  $|A| < k - 1$  then
8:      $A[j] \leftarrow 1$ 
9:   else if  $A[j] = 0$  and  $|A| = k - 1$  then
10:    for all  $j' \in A$  do  $\triangleright$  Loop over all  $j'$  s.t.  $A[j'] > 0$ 
11:       $A[j'] \leftarrow A[j'] - 1$   $\triangleright$  If  $A[j']$  reaches 0, remove it
```

Output: On query $j \in [n]$, **return** $A[j]$

First example: the Misra-Gries algorithm, alternative view (2/4)

Input: Parameter $\varepsilon \in (0, 1]$

```
1:  $A \leftarrow \emptyset$ 
2: Set  $k \leftarrow \lceil 1/\varepsilon \rceil$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i = j \in [n]$ 
5:   if  $A[j] > 0$  then
6:      $A[j] \leftarrow A[j] + 1$ 
7:   else if  $A[j] = 0$  and  $|A| < k - 1$  then
8:      $A[j] \leftarrow 1$ 
9:   else if  $A[j] = 0$  and  $|A| = k - 1$  then
10:     for all  $j' \in A$  do           ▷ Loop over all  $j'$  s.t.  $A[j'] > 0$ 
11:        $A[j'] \leftarrow A[j'] - 1$    ▷ If  $A[j']$  reaches 0, remove it
```

Output: On query $j \in [n]$, **return** $A[j]$

First example: the Misra-Gries algorithm (3/4)

Theorem 39. *The MISRA-GRIES algorithm is a deterministic one-pass algorithm which, for any given parameter $\varepsilon \in (0, 1]$, provides $\hat{f}_1, \dots, \hat{f}_n$ of all element frequencies such that*

$$f_j - \varepsilon m \leq \hat{f}_j \leq f_j, \quad j \in [n]$$

with space complexity $s = O(\log(mn)/\varepsilon)$. (In particular, it can be used to solve the MAJORITY problem in two passes.)

First example: the **Misra-Gries** algorithm, solving Majority (4/4)

Second example: Approximate Counting

Second example: Approximate Counting and the Morris Counter

```
1:  $x \leftarrow 0$ 
2: for all  $1 \leq i \leq m$  do
3:   Get item  $a_i \in \{0, 1\}$ 
4:   if  $a_i = 1$  then
5:      $r_i \leftarrow \text{Bern}(1/2^x)$        $\triangleright$  Independent of previous choices.
6:      $x \leftarrow x + r_i$ 
7: return  $\hat{d} \leftarrow 2^x - 1$ 
```

Second example: Approximate Counting and the Morris Counter

```
1:  $C_0 \leftarrow 1$ 
2: for all  $1 \leq i \leq m$  do
3:   Get item  $a_i \in \{0, 1\}$ 
4:   if  $a_i = 1$  then
5:      $r_i \leftarrow \text{Bern}(1/C_{i-1})$      $\triangleright$  Independent of previous choices.
6:   else  $r_i \leftarrow 0$ 
7:    $C_i \leftarrow 2^{r_i} C_{i-1}$ 
8: return  $\hat{d} \leftarrow C_m - 1$ 
```

Throwback: Law of Total Expectation (and Friends)

Second example: the Morris Counter (1/3)

```
1:  $C_0 \leftarrow 1$ 
2: for all  $1 \leq i \leq m$  do
3:   Get item  $a_i \in \{0, 1\}$ 
4:   if  $a_i = 1$  then
5:      $r_i \leftarrow \text{Bern}(1/C_{i-1})$     ▷ Independent of previous choices.
6:   else  $r_i \leftarrow 0$ 
7:    $C_i \leftarrow 2^{r_i} C_{i-1}$ 
8: return  $\hat{d} \leftarrow C_m - 1$ 
```

Second example: the Morris Counter (2/3)

```
1:  $C_0 \leftarrow 1$ 
2: for all  $1 \leq i \leq m$  do
3:   Get item  $a_i \in \{0, 1\}$ 
4:   if  $a_i = 1$  then
5:      $r_i \leftarrow \text{Bern}(1/C_{i-1})$     ▷ Independent of previous choices.
6:   else  $r_i \leftarrow 0$ 
7:    $C_i \leftarrow 2^{r_i} C_{i-1}$ 
8: return  $\hat{d} \leftarrow C_m - 1$ 
```

Second example: the Morris Counter (3/3)

Median-of-Means!

If some random variable X has the "correct" expectation μ but "bad" variance σ^2 , by taking the average over k independent repetitions, the expectation stays correct:

$$\mathbb{E}[\bar{X}] = \frac{1}{k} \sum_{i=1}^k \mathbb{E}[X_i] = \frac{1}{k} \sum_{i=1}^k \mu = \mu$$

But the variance goes down!

$$\text{Var}[\bar{X}] = \frac{1}{k^2} \sum_{i=1}^k \text{Var}[X_i] = \frac{1}{k^2} \sum_{i=1}^k \sigma^2 = \frac{\sigma^2}{k}$$

💡 Take $k = O(1/\varepsilon^2)$ to get good accuracy with **Chebyshev** (constant proba), and then repeat the whole thing $T = O(\log(1/\delta))$ times to get proba $1 - \delta$ with **median trick**.

Second example: the Morris Counter, Median-of-Means

Theorem 40. *The medians-of-means version of the MORRIS COUNTER is a randomised one-pass algorithm which, for any given parameters $\epsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of non-zero elements of the stream such that*

$$\Pr \left[(1 - \epsilon)d \leq \hat{d} \leq (1 + \epsilon)d \right] \geq 1 - \delta$$

with space complexity

$$s = O \left(\frac{\log \log m}{\epsilon^2} \cdot \log \frac{1}{\delta} \right)$$

that is, doubly logarithmic in m .

Did we need to do that?

No.

No need for median-of-means here! Can modify the Morris Counter to directly get this high-accuracy guarantee, via a slightly more **careful version**.

And we even get **better space complexity**.

Second example: the Morris Counter, careful version (1/2)

Second example: the Morris Counter, careful version (2/2)

Theorem 41. *The “careful” version of MORRIS COUNTER is a randomised one-pass algorithm which, for any given parameters $\epsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of non-zero elements of the stream such that*

$$\Pr \left[(1 - \epsilon)d \leq \hat{d} \leq (1 + \epsilon)d \right] \geq 1 - \delta$$

with space complexity

$$s = O \left(\log \log m + \log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right)$$

that is, doubly logarithmic in m and logarithmic in $1/\epsilon$.

Third example: Distinct Elements

Third example: Distinct Elements, the TideMark (AMS) algorithm (1/5)

```
1: Pick  $h: [n] \rightarrow [n]$  from a strongly universal hashing family
2:  $z \leftarrow 0$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i \in [n]$ 
5:   if  $\text{zeros}(h(a_i)) \geq z$  then
6:      $z \leftarrow \text{zeros}(h(a_i))$ 
7: return  $\sqrt{2} \cdot 2^z$ 
```

Third example: Distinct Elements, the Tdemark (AMS) algorithm (2/5)



```
1: Pick  $h: [n] \rightarrow [n]$  from a strongly universal hashing family
2:  $z \leftarrow 0$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i \in [n]$ 
5:   if  $\text{zeros}(h(a_i)) \geq z$  then
6:      $z \leftarrow \text{zeros}(h(a_i))$ 
7: return  $\sqrt{2} \cdot 2^z$ 
```

Third example: Distinct Elements, the Tdemark (AMS) algorithm (3/5)

```
1: Pick  $h: [n] \rightarrow [n]$  from a strongly universal hashing family
2:  $z \leftarrow 0$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i \in [n]$ 
5:   if  $\text{zeros}(h(a_i)) \geq z$  then
6:      $z \leftarrow \text{zeros}(h(a_i))$ 
7: return  $\sqrt{2} \cdot 2^z$ 
```

Third example: Distinct Elements, the Tdemark (AMS) algorithm (4/5)

```
1: Pick  $h: [n] \rightarrow [n]$  from a strongly universal hashing family
2:  $z \leftarrow 0$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i \in [n]$ 
5:   if  $\text{zeros}(h(a_i)) \geq z$  then
6:      $z \leftarrow \text{zeros}(h(a_i))$ 
7: return  $\sqrt{2} \cdot 2^z$ 
```

Third example: Distinct Elements, the Tidemark (AMS) algorithm (5/5)

Theorem 42. *The (median trick version of the) TIDEMARK (AMS) algorithm is a randomised one-pass algorithm which, for any given parameter $\delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of distinct elements of the stream such that, for some absolute constant $C > 0$,*

$$\Pr \left[\frac{1}{C} \cdot d \leq \hat{d} \leq C \cdot d \right] \geq 1 - \delta$$

with space complexity

$$s = O \left(\log n \cdot \log \frac{1}{\delta} \right).$$

Can we do better?

Yes.

Third example: Distinct Elements, the BJKST algorithm (1 / 4)

Input: Parameter $\varepsilon \in (0, 1]$

1: Set $k \leftarrow O(\log^2 n / \varepsilon^4)$, $T \leftarrow \Theta(1 / \varepsilon^2)$

2: Pick $h: [n] \rightarrow [n]$ from a strongly universal hashing family

3: Pick $g: [n] \rightarrow [k]$ from a strongly universal hashing family

4: $z \leftarrow 0$, $B \leftarrow \emptyset$

5: **for all** $1 \leq i \leq m$ **do**

6: Get item $a_i \in [n]$

7: **if** $\text{zeros}(h(a_i)) \geq z$ **then**

8: $B \leftarrow B \cup \{(g(a_i), \text{zeros}(h(a_i)))\}$

9: **while** $|B| \geq T$ **do**

10: $z \leftarrow z + 1$

11: Remove every (a, b) with $b < z$ from B

12: **return** $|B| \cdot 2^z$

Third example: Distinct Elements, the BJKST algorithm (2/4)

Input: Parameter $\varepsilon \in (0, 1]$

- 1: Set $k \leftarrow O(\log^2 n / \varepsilon^4)$, $T \leftarrow \Theta(1 / \varepsilon^2)$
- 2: Pick $h: [n] \rightarrow [n]$ from a strongly universal hashing family
- 3: Pick $g: [n] \rightarrow [k]$ from a strongly universal hashing family

- 4: $z \leftarrow 0$, $B \leftarrow \emptyset$
 - 5: **for all** $1 \leq i \leq m$ **do**
 - 6: Get item $a_i \in [n]$
 - 7: **if** $\text{zeros}(h(a_i)) \geq z$ **then**
 - 8: $B \leftarrow B \cup \{(g(a_i), \text{zeros}(h(a_i)))\}$
 - 9: **while** $|B| \geq T$ **do**
 - 10: $z \leftarrow z + 1$
 - 11: Remove every (a, b) with $b < z$ from B
 - 12: **return** $|B| \cdot 2^z$
-

Third example: Distinct Elements, the BJKST algorithm (3/4)

Input: Parameter $\varepsilon \in (0, 1]$

- 1: Set $k \leftarrow O(\log^2 n / \varepsilon^4)$, $T \leftarrow \Theta(1 / \varepsilon^2)$
 - 2: Pick $h: [n] \rightarrow [n]$ from a strongly universal hashing family
 - 3: Pick $g: [n] \rightarrow [k]$ from a strongly universal hashing family
 - 4: $z \leftarrow 0$, $B \leftarrow \emptyset$
 - 5: **for all** $1 \leq i \leq m$ **do**
 - 6: Get item $a_i \in [n]$
 - 7: **if** $\text{zeros}(h(a_i)) \geq z$ **then**
 - 8: $B \leftarrow B \cup \{(g(a_i), \text{zeros}(h(a_i)))\}$
 - 9: **while** $|B| \geq T$ **do**
 - 10: $z \leftarrow z + 1$
 - 11: Remove every (a, b) with $b < z$ from B
 - 12: **return** $|B| \cdot 2^z$
-

Third example: Distinct Elements, the BJKST algorithm (4/4)

Theorem 43. *The (median trick version of the) BJKST algorithm is a randomised one-pass algorithm which, for any given parameters $\epsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of distinct elements of the stream such that, for some absolute constant $C > 0$,*

$$\Pr \left[(1 - \epsilon) \cdot d \leq \hat{d} \leq (1 + \epsilon)d \right] \geq 1 - \delta$$

with space complexity

$$s = O \left(\left(\log n + \frac{\log(1/\epsilon) + \log \log n}{\epsilon^2} \right) \cdot \log \frac{1}{\delta} \right).$$

... Can we do better?

Yes. (A little bit)

But it's a much more complicated algorithm/analysis. We will not see it here.